Statistical Computing

Compiled Revision Notes

Table of Contents

Week 1: Fundamental Concepts & Discrete Distributions **5**
- Definitions . . . 5
- Measures of Centrality and Variation . . . 5
- Probability Theory Basics . . . 6
- Conditional Probability and Independence . . . 6
- Advanced Probability Theorems . . . 7
- Discrete Random Variables . . . 7
- Cumulative Distribution Function (CDF) . . . 8
- Discrete Probability Distributions . . . 8

Week 2: Continuous, Sampling & Hypothesis Testing **9**
- Continuous Random Variables . . . 9
- The Normal Distribution . . . 9
- Sampling Distributions . . . 10
- Hypothesis Testing Basics . . . 10
- Confidence Intervals . . . 11
- Hypothesis Tests for Proportions . . . 12
- One Sample t-Test . . . 13
- Comparing Two Means: Independent Samples . . . 13
- Comparing Two Means: Paired Samples . . . 15
- Non-Parametric Alternatives: Wilcoxon Tests . . . 15

Week 3: Enumerative Data Analysis & MLE **17**
- Enumerative Data Analysis (Chi-Squared) . . . 17
- Chi-Squared Goodness-of-Fit Test . . . 17
- M&Ms Example . . . 18
- Chi-Squared Test of Independence . . . 18
- Maximum Likelihood Estimation . . . 21
- MLE Examples . . . 22
- R Implementation . . . 23

Week 4: Advanced MLE & Numerical Methods **23**
- Complex MLE & The Need for Optimization . . . 23
- Numerical Optimization Methods . . . 23
- MLE Optimization in R . . . 24
- Why We Love MLE (Theoretical Properties) . . . 25
- The Likelihood Ratio Test (LRT) . . . 25

Week 5: Generalised Linear Models **26**
- Linear Regression Review and Diagnostics . . . 26
- Limitations of Linear Regression . . . 26
- Introduction to Generalised Linear Models (GLMs) . . . 27
- Logistic Regression . . . 27

Nested Models and Interactions . . . 28
Fitting GLMs . . . 28

Week 6: Generalised Linear Models (Continued) 29
Model Evaluation and Deviance . . . 29
Model Comparison Tools . . . 29
Poisson Regression for Count Data . . . 30
Handling Categorical Predictors in R . . . 30
Variance Functions and Overdispersion . . . 31
Dealing with Overdispersion . . . 31
Positive Continuous Responses (Gamma GLM) . . . 32
Quick Reference: Coefficient Interpretation . . . 32

Week 7: Density Estimation 33
Motivation and Introduction to Density Estimation . . . 33
Histograms and Local Averages . . . 33
Kernel Density Estimation (KDE) . . . 33
Computational Complexity . . . 35
Applications (Bayesian Classification) . . . 36
Multivariate Densities . . . 36
Collinearity Review . . . 36

Week 8: Mixtures 37
Motivation . . . 37
Mixture Distributions . . . 37
Estimating the Parameters . . . 37
The Expectation-Maximization (EM) Algorithm . . . 38
Number of Components . . . 39
Mixture of Regressions . . . 39
Multivariate Mixtures . . . 39

Week 9: Markov Chains 40
Motivation: Beyond Independence . . . 40
The Markov Property . . . 40
Transition Matrices . . . 40
Multi-Step Transitions . . . 41
Structure of Markov Chains . . . 42
Stationary Distribution . . . 42
Ergodic Theorem . . . 43
Computing the Stationary Distribution . . . 43
First Passage Time and Expected Passage Time . . . 44
Mean Return Time . . . 44
Maximum Likelihood Estimation (MLE) of Markov Chains . . . 44
R Implementation (markovchain package) . . . 45
Applications of Markov Chains . . . 45

Week 10: Bayesian Inference 46
Motivation: Frequentist vs Bayesian . . . 46

Bayes Theorem for Inference . . . 46
Conjugacy and Conjugate Priors . . . 46
The Beta-Binomial Model . . . 47
Prior - Likelihood - Posterior Relationship . . . 47
Example: Factory Defect Rate . . . 48
Bayesian Point Estimates and Intervals . . . 48
R Implementation . . . 49

Week 11: Bayesian Inference and MCMC **49**
The Intractability Problem . . . 49
Grid Approximation . . . 49
Sampling from the Posterior . . . 50
Markov Chain Monte Carlo (MCMC) . . . 50
The Metropolis-Hastings Algorithm . . . 50
Proposal Tuning and Step Size . . . 50
Assessing Convergence and Diagnostics . . . 51
Applied Bayesian Workflow . . . 52
Prior Influence . . . 53

Week 12: Hidden Markov Models **53**
Motivation and Concept . . . 53
Model Definition and Structure . . . 53
Mixture Models and Time Dependence . . . 54
Likelihood and The Expectation-Maximization (EM) Algorithm . . . 54
Model Selection and Posterior State Decoding . . . 55
Complete Exam Quick Reference Table . . . 55

Week 1: Fundamental Concepts & Discrete Distributions

Definitions

- **Population:** The complete collection of all individuals or items under consideration in a statistical study.
- **Sample:** A subset of the population from which information is actually collected.

Parameters vs Statistics

- **Population Parameters** (constants, usually unknown):
  - $\mu \rightarrow$ population mean
  - $\sigma \rightarrow$ population standard deviation
- **Sample Statistics** (random variables):
  - $\bar{x} \rightarrow$ sample mean
  - $s \rightarrow$ sample standard deviation

Measures of Centrality and Variation

Measures of Centrality (Centre)

1. **Mean ($\bar{x}$):** Arithmetic average

$$\bar{x} = \frac{\sum x_i}{n}$$

2. **Median:** Middle value when data is ordered.
3. **Mode:** Most frequently occurring value.

Measures of Variation (Spread)

- **Range:** Max $-$ Min. Very sensitive to outliers.
- **Sample Variance (s^2):**

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$$

- **Sample Standard Deviation (s):**

$$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$$

Probability Theory Basics

Core Rules

- **Sample Space (Ω):** Set of all possible outcomes. $P(\Omega) = 1$
- **Empty Set ($\emptyset$):** Impossible event. $P(\emptyset) = 0$
- **Probability Bounds:** $0 \leq P(A) \leq 1$
- **Complement Rule:** $P(A^c) = 1 - P(A)$. Notation: A^c, $\bar{A}$, or A'.

Combining Events

- **Intersection (AND):** $A \cap B$
- **Union (OR):** $A \cup B$

Addition Rules

- **General Addition Rule:** $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- **Disjoint (Mutually Exclusive) Events:**
  - Cannot occur together
  - $A \cap B = \emptyset$
  - $P(A \cup B) = P(A) + P(B)$

Conditional Probability and Independence

Conditional Probability

Probability that B occurs given that A has occurred:

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)}$$

Multiplication Law

$$P(A \cap B) = P(B \mid A) \times P(A)$$

Independence

Two events are independent if one does not affect the other:

- $P(B \mid A) = P(B)$, or
- $P(A \cap B) = P(A) \times P(B)$

Advanced Probability Theorems

Bayes' Theorem

Used to reverse conditional probabilities:

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

Law of Total Probability

If $A_1, A_2, \ldots, A_n$ partition the sample space:

$$P(B) = \sum_{i=1}^{n} P(B \mid A_i) \times P(A_i)$$

Discrete Random Variables

Definition

A **Random Variable (X)** is a numerical model for a measurement.

- **Discrete RV:** Takes a finite or countably infinite number of values.
- **Bernoulli RV:** Simplest discrete RV.
  Takes value:
  - 1 for success
  - 0 for failure

Probability Mass Function (pmf)

$$f(x) = P(X = x)$$

Expected Value (Mean)

The long-run average or centre of gravity:

$$E(X) = \mu = \sum x \cdot P(X = x)$$

Example (Fair die):

$$E(X) = 3.5$$

Cumulative Distribution Function (CDF)

$$F(x) = P(X \leq x)$$

- For discrete RVs, the CDF has a **step shape**.
- **At least rule:** $P(X \geq k) = 1 - P(X < k)$

Discrete Probability Distributions

Binomial Distribution

Used for the number of successes in n trials.

Assumptions (Always state in exams)

1. Fixed number of trials (n)
2. Constant probability of success (p)
3. Trials are independent

Model

$$P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}$$

Parameters

- **Mean:** $\mu = np$
- **Standard Deviation:**

$$\sigma = \sqrt{np(1-p)}$$

Poisson Distribution

Used for counting arrivals in a fixed interval of time or space.

Assumptions

1. Probability proportional to interval size
2. Probability of two or more arrivals in a very small interval is negligible
3. Non-overlapping intervals are independent

Model

$$P(X = x) = \frac{e^{-\alpha t}(\alpha t)^x}{x!}$$

- α = average rate per unit
- t = length of interval

Key Property (Very Exam Important)

$$Rate = \lambda = E(X) = Var(X) = \alpha t$$

Week 2: Continuous, Sampling & Hypothesis Testing

Continuous Random Variables

Definition

A continuous random variable can take values anywhere in a continuum, such as height, temperature, or sales.

- **Density Function ($f(x)$):**
  A curve where the area under the curve between two points represents probability.
- **Total Area:**
  The total area under $f(x)$ is always 1:

$$\int_{-\infty}^{\infty} f(x)\,dx = 1$$

Uniform Distribution

The simplest continuous distribution where probability is constant between a and b.

- **PDF:**

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

The Normal Distribution

Properties

- Defined by **Mean (μ)** and **Variance (σ^2)**.
- Notation: $X \sim N(\mu, \sigma^2)$

Empirical Rule (68, 95, 99.7)

- 68% of data lies within $\mu \pm 1\sigma$
- 95% of data lies within $\mu \pm 2\sigma$
- 99.7% of data lies within $\mu \pm 3\sigma$

Sampling Distributions

Central Limit Theorem (CLT)

Regardless of the population distribution, if sample size n is large, the distribution of the sample mean $\bar{X}$ is approximately normal.

- **Mean of $\bar{X}$:** $E(\bar{X}) = \mu$
- **Variance of $\bar{X}$:**

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

- **Standard Error:** $\frac{\sigma}{\sqrt{n}}$
- **Z Statistic:**

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}}$$

Hypothesis Testing Basics

Quick Reference: Which Test Should I Use?

Decision Tree for Selecting the Appropriate Test:

Scenario	Test to Use	R Command	Key Assumption
One sample, testing mean, normal data	One-sample t-test	`t.test(x, mu = mu0)`	Data approximately normal (Shapiro-Wilk $p > 0.05$)
One sample, testing mean, NOT normal	Wilcoxon Signed-Rank	`wilcox.test(x, mu = mu0)`	Data symmetric (doesn't require normality)
Two paired samples, normal differences	Paired t-test	`t.test(x, y, paired = TRUE)`	Differences approximately normal
Two paired samples, NOT normal differences	Wilcoxon Signed-Rank (paired)	`wilcox.test(x, y, paired = TRUE)`	Symmetric differences
Two independent samples, both normal, equal variances	Two-sample t-test (Student's)	`t.test(x, y, var.equal = TRUE)`	Both groups normal; check with Levene's test
Two independent samples, normal, unequal variances	Two-sample t-test (Welch's)	`t.test(x, y, var.equal = FALSE)`	Both groups normal; Welch's doesn't assume equal variance

Scenario	Test to Use	R Command	Key Assumption
Two independent samples, NOT normal	Wilcoxon Rank Sum	`wilcox.test(x, y)`	Doesn't require normality
Categorical data, goodness-of-fit	Chi-squared Goodness-of-Fit	`chisq.test(x, p = probs)`	Expected counts ≥ 5 in 80% of cells (Rule of 5)
Categorical data, independence of two factors	Chi-squared Test of Independence	`chisq.test(table)`	Expected counts ≥ 5 in 80% of cells
Categorical data, small expected counts	Fisher's Exact Test	`fisher.test(table)`	No cell assumptions; exact p-value
Comparing proportions between groups	Chi-squared or Fisher's Exact	See above	Depends on sample sizes

Core Concepts

- **Null Hypothesis (H_0):**
  Assumed true. Always contains equality ($=$, $\leq$, $\geq$).
- **Alternative Hypothesis (H_1):**
  The claim we seek evidence for. Always contains inequality ($\neq$, $<$, $>$).

Errors

- **Type I Error (α):**
  Rejecting H_0 when it is actually true.
- **Type II Error (β):**
  Failing to reject H_0 when it is actually false.

The p-value

The probability of observing a result at least as extreme as the one obtained, assuming H_0 is true.

- **Decision Rule:**
  Reject H_0 if

$$\text{p-value} < \alpha$$

Confidence Intervals

Definition

An interval constructed around $\bar{x}$ where we are reasonably confident the true population mean μ lies.

- **Interpretation:**
  In repeated sampling, 95% of such intervals would contain μ.

Formula (Known σ or Large n)

$$\bar{x} \pm z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$

Example: Cola Cans

- $\bar{x} = 299.64$
- $n = 100$
- $\sigma = 1.2$

Resulting interval:

$$[299.40, 299.88]$$

Since 300 is not in the interval, reject H_0.

Hypothesis Tests for Proportions

Used for categorical data.

Two Approaches

Test	Method	Best For
`prop.test`	Normal (χ^2) approximation	Large samples ($n \geq 30$)
`binom.test`	Exact binomial probabilities	Small samples or when exact results are needed

Example: Thanos Snap

- $H_0 : p = 0.5$
- $H_1 : p \neq 0.5$
- Observed: 64 vanished out of 100

R Code (Approximate):

```
prop.test(64, 100, p = 0.5)
```

- p-value = 0.0069
  Reject H_0.

R Code (Exact - preferred for small samples):

```
binom.test(64, 100, p = 0.5)
```

- p-value = 0.0105
  Reject H_0.

`binom.test` Arguments

```r
binom.test(x, n, p = 0.5, alternative = "two.sided")
```

- `x` - number of successes observed
- `n` - number of trials
- `p` - hypothesised probability of success under H_0
- `alternative` - `"two.sided"`, `"less"`, or `"greater"`

When to Use Each

- Use **`binom.test`** when n is small (roughly $n < 30$), or when you need exact p-values.
- Use **`prop.test`** for large samples; it also supports comparing two proportions (`prop.test(c(x1, x2), c(n1, n2))`).

One Sample t-Test

Used when population variance σ^2 is unknown.

- Uses Student's t distribution
- Degrees of freedom: $df = n - 1$

Assumptions

1. Data is numeric and continuous.
2. Data is normally distributed.

Normality Test: Shapiro-Wilk

- If p-value > 0.05, assume normality.

Example: Corrib River Radiation

- $H_0 : \mu \geq 5$
- $H_1 : \mu < 5$

R Code:

```r
t.test(corrib, mu = 5, alternative = "less")
```

- p-value $= 0.002$
  Reject H_0. Water is safe.

Comparing Two Means: Independent Samples

Used to compare two separate groups.

Steps

1. **Check Normality:**
   Shapiro-Wilk test on both groups.
   - If $p > 0.05$ in both groups: data appears normal, proceed to step 2
   - If $p < 0.05$ in either group: data is not normal, **use Wilcoxon test instead**
2. **Check Variances (Parametric Only):**
   - **Levene's Test** (more robust, doesn't assume normality)

     ```
     leveneTest(y ~ group, data = df)
     ```

   - **Bartlett's Test** (assumes normality, avoid if data not perfectly normal)

     ```
     bartlett.test(list(group1, group2))
     ```

   - If p-value > 0.05: variances appear equal, use **Pooled t-test** (`var.equal = TRUE`)
   - If p-value < 0.05: variances are unequal, use **Welch t-test** (`var.equal = FALSE`)
3. **Run t-Test:**
   Choose the appropriate variant based on the variance test result.

When t-Test Assumptions Fail

Problem	Diagnostic	Solution
Data not normal	Shapiro-Wilk $p < 0.05$	Use **Wilcoxon test**
Variances very unequal	Levene $p < 0.05$ and ratio $> 3{:}1$	Use **Welch t-test** (default in R)
Outliers present	Visualize with boxplot	Use **Wilcoxon test** or remove outliers
Small sample + uncertainty	$n < 30$ and Shapiro-Wilk p borderline	Use **Wilcoxon test** for safety
Ordinal/ranked data	Data is ranks or categories	Use **Wilcoxon test**

Default Recommendation: R's default `t.test()` uses **Welch's test** (no equal variance assumption), which is safe in most cases. If data fails normality, switch to `wilcox.test()`.

Which t-Test to Use

Situation	Test	R Code
Unequal variances (or unsure)	Welch Two Sample t-test - **does NOT assume equal variances**	`t.test(x, y, alternative = ...)`

Situation	Test	R Code
Equal variances confirmed	Pooled (Student's) t-test - assumes equal variances	`t.test(x, y, var.equal = TRUE, alternative = ...)`

Default in R: `t.test()` uses Welch's test (`var.equal = FALSE`) - safe to use in all cases.

R Code (Welch - unequal/unknown variances):

```r
t.test(x, y, alternative = "less")
```

R Code (Pooled - equal variances confirmed):

```r
t.test(x, y, var.equal = TRUE, alternative = "less")
```

Comparing Two Means: Paired Samples

Used when observations are dependent or matched.

Logic

Performs a one sample t-test on the differences between paired observations.

Example: Diet Study

- $H_1 : \mu_{\text{diff}} > 0$

R Code:

```r
t.test(before, after, paired = TRUE, alternative = "greater")
```

- p-value = 0.02
  Reject H_0. Diet worked.

Warning

Using an independent t-test on paired data is incorrect and can increase the chance of a Type II error.

Non-Parametric Alternatives: Wilcoxon Tests

When to Use Wilcoxon Tests

Wilcoxon tests are **non-parametric alternatives** to t-tests and should be used when:

- **Data is not normally distributed** (Shapiro-Wilk $p < 0.05$)
- **Data is ordinal** (ranks or ordered categories, not measurements)
- **Sample size is small** ($n < 30$ and normality questionable)

- **Outliers are present** that cannot be removed
- **Data has extreme skewness**

Key Difference: t-tests compare **means**; Wilcoxon tests compare **medians** (or distributions more generally).

Wilcoxon Rank Sum Test (Independent Samples)

Purpose: Tests whether two independent groups have the same median.

Hypotheses:

- H_0: The two groups have the same distribution (or median)
- H_1: The two groups have different distributions (or medians)

How it works:

1. Combine both samples and rank all values from smallest to largest
2. Calculate the sum of ranks for each group
3. Compare the rank sums to determine if they differ significantly

R Code:

```r
# Independent samples
wilcox.test(group1, group2, alternative = "two.sided")

# One-sided alternatives
wilcox.test(group1, group2, alternative = "less")     # group1 median < group2
 ↪  median
wilcox.test(group1, group2, alternative = "greater")  # group1 median > group2
 ↪  median
```

Wilcoxon Signed-Rank Test (Paired Samples)

Purpose: Tests whether paired observations have a median difference of zero (equivalent to paired t-test but non-parametric).

How it works:

1. Calculate differences between paired observations
2. Rank the absolute values of these differences
3. Assign + or - sign to ranks based on whether the difference is positive or negative
4. Compare signed rank sums

R Code:

```r
# Paired samples
wilcox.test(before, after, paired = TRUE, alternative = "two.sided")

# One-sided: test if after > before
wilcox.test(before, after, paired = TRUE, alternative = "greater")
```

t-Test vs Wilcoxon: Summary

Aspect	t-Test	Wilcoxon
Data Type	Continuous, normal	Any distribution (especially non-normal)
Compares	Means	Medians/distributions
Assumptions	Normality required	No normality assumption
Small samples	Risky if non-normal	Preferred
Power	Higher (if normal)	Lower but reliable
Outliers	Affected	Robust
Use When	Confident data is normal	Any doubt about normality

Week 3: Enumerative Data Analysis & MLE

Enumerative Data Analysis (Chi-Squared)

Qualitative vs Quantitative

Previously we analysed **quantitative data** (height, weight, marks).

Now we analyse **qualitative (categorical) data**:

- Data consists of **counts / frequencies**
- Examples: Eye colour, Yes/No, Defective/Not defective

We compare **Observed vs Expected frequencies**.

The Chi-Squared Distribution (χ^2)

- Not symmetric
- Right skewed
- Range: $0 \to \infty$
- Depends on **degrees of freedom (df)**
- As df increases, it becomes more Normal shaped
- Right tail area = significance level α

Chi-Squared Goodness-of-Fit Test

Purpose

Tests whether observed categorical data matches a claimed distribution.

Test Statistic

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

Where:

- O = Observed frequency
- E = Expected frequency
- $df = k - 1$

Large χ^2 means observed differs strongly from expected.

M&Ms Example

Claim (H_0):

30% Brown, 20% Yellow, 20% Red, 10% Orange, 10% Green, 10% Blue

Hypotheses:

- H_0: Distribution matches claim
- H_1: Distribution differs

Reject H_0 if $\chi^2_{calc} > \chi^2_{critical}$.

R Code

```
chocolate <- c(67, 36, 43, 24, 23, 7)
probs <- c(0.3, 0.2, 0.2, 0.1, 0.1, 0.1)
chisq.test(chocolate, p = probs)
```

Chi-Squared Test of Independence

Purpose

Tests whether two categorical variables are related.

Hypotheses

- H_0: Variables are independent
- H_1: Variables are dependent

Expected Counts Formula

For contingency table:

$$E_{ij} = \frac{(\text{Row Total})(\text{Column Total})}{\text{Grand Total}}$$

Degrees of freedom:

$$df = (r-1)(c-1)$$

Assumptions & Requirements

1. **Categorical variables** (not continuous)
2. **Independent observations** (each row in contingency table is from a different subject)
3. **Rule of 5 (Critical Requirement):**
   - At least **80% of expected counts** ≥ 5
   - **No expected count** < 1

Why Rule of 5 Matters

- When expected counts are too small, the chi-squared distribution is not a good approximation
- This leads to inaccurate p-values (can be either too large or too small)
- Violating this assumption can cause incorrect conclusions

What to Do if Rule of 5 Violated

1. **Combine categories:** Merge categories with small expected counts
   - Example: If "Other" has expected count < 5, merge it with the most similar category
   - This reduces dimensionality but preserves information
2. **Use Fisher's Exact Test:** For 2 x 2 tables, Fisher's test computes exact p-values without relying on the chi-squared approximation

   ```
   fisher.test(contingency_table)
   ```
3. **Collect more data:** Larger sample sizes increase expected counts

Checking Rule of 5 in R

```r
# After running chi-squared test
result <- chisq.test(data)
result$expected  # View expected counts

# Check if any expected < 5
sum(result$expected < 5)  # If > 0, violation detected

# Proportion of expected counts < 5
mean(result$expected < 5)  # If > 0.2 (20%), violation
```

Fisher's Exact Test

Used when chi-squared test assumptions are violated (Rule of 5 fails).

- Computes exact binomial probabilities instead of relying on chi-squared approximation
- Ideal for 2 x 2 contingency tables with small expected counts
- Can be slow for large tables

R Code:

```
fisher.test(contingency_table)
```

When to use: * Rule of 5 is violated ($> 20\%$ of expected counts < 5) * Small sample sizes * Conservative p-values preferred

Effect Size: Statistical vs Practical Significance

The Problem with Large Samples

- **Statistical significance** tells you if a difference exists.
- **Practical importance** tells you if the difference matters.
- With very large n, even tiny differences can produce small p values.
- Example: A 2 second improvement may be statistically significant but practically useless.

The Solution: Effect Size Effect size measures the **magnitude** of a difference.

Chi-Squared Tests: Phi Coefficient For 2×2 tables:

$$\phi = \sqrt{\frac{\chi^2}{n}}$$

Guidelines:

- 0.1 small
- 0.3 medium
- 0.5 large

---

t Tests: Cohen's d Used when comparing two means.

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s}$$

For independent samples, use the pooled standard deviation.

Guidelines:

- 0.2 small
- 0.5 medium
- 0.8+ large

Maximum Likelihood Estimation

The Core Idea

How do we find the "best" parameters such as μ or λ?

MLE finds the parameter that makes your data most likely.

- **Fisher's Principle:** Choose parameter θ that makes the observed data most probable.
- Goal: Find θ that maximizes $P(\text{data} \mid \theta)$ i.e. the Likelihood Function $L(\theta)$.

MLE Step by Step

Likelihood Function Write the probability of the entire dataset.

If observations are independent:

$$L(\theta) = \prod f(x_i \mid \theta)$$

Log-Likelihood Take the natural log:

$$\ell(\theta) = \sum \ln\left(f(x_i \mid \theta)\right)$$

Why?

- Differentiating a product is messy
- Differentiating a sum is easier
- Logs turn products into sums

Differentiate Find derivative with respect to θ:

$$\frac{d\ell}{d\theta}$$

Solve Set derivative equal to 0 and solve for θ.

This gives the MLE estimate.

MLE Examples

Poisson Distribution (Horse Kicks)

- Data: Counts of deaths by horse kicks (von Bortkiewicz data)
- Model:

$$X \sim \text{Poisson}(\lambda)$$

MLE Result

$$\hat{\lambda}_{MLE} = \frac{1}{n}\sum x_i = \bar{x}$$

Takeaway: For Poisson, the MLE for λ is the **sample mean**.

Normal Distribution

We estimate two parameters: μ and σ^2.

Estimating the Mean

$$\hat{\mu} = \bar{x}$$

Takeaway: MLE mean equals the sample mean.

Estimating the Variance

$$\hat{\sigma}^2_{MLE} = \frac{1}{n}\sum (x_i - \bar{x})^2$$

Bias Issue

- MLE divides by $n \rightarrow$ biased (underestimates variance)
- Sample variance:

$$s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$$

Uses Bessel's correction and is unbiased.

Conclusion: For large n, difference is negligible.

R Implementation

For complex models, solve numerically.

Note: R minimizes functions, so use the **negative log-likelihood**.

```
library(stats4)

# 1. Define Negative Log-Likelihood
nloglik <- function(lambda) {
    return(-sum(dpois(data, lambda, log = TRUE)))
}

# 2. Run Optimizer
fit <- mle(nloglik, start = list(lambda = 1))
summary(fit)
```

Week 4: Advanced MLE & Numerical Methods

Complex MLE & The Need for Optimization

When Math Fails (The Gamma Distribution)

- The Gamma distribution models right-skewed data, for example insurance claims.
- It uses two parameters: α (shape) and β (scale).
- **The Problem:** When you take the derivative of the Gamma log-likelihood and set it equal to 0, there is no simple closed-form solution. You cannot solve it by hand.
- **The Solution:** Numerical optimization. We use a computer to find where the derivative is approximately zero, which corresponds to the peak of the likelihood.

Numerical Optimization Methods

When we cannot find the maximum likelihood mathematically, we use algorithms to walk uphill to the peak.

Gradient Ascent / Descent

- **How it works:** Finds the direction of the steepest slope, the gradient, and takes a step in that direction.
- **Pros:** Simple to implement; only needs first derivatives.
- **Cons:** Slow, linear convergence; choosing the right step size is tricky.

Newton's Method

- **How it works:** Uses curvature, the Hessian matrix of second derivatives, to fit a quadratic curve and jump straight to its maximum.
- **Pros:** Very fast, quadratic convergence; fewer, smarter steps.
- **Cons:** Fails if the Hessian matrix is not invertible or near saddle points; computationally expensive because it requires second derivatives.

BFGS (Quasi-Newton)

- **How it works:** Achieves Newton-like speed without computing second derivatives. It approximates the Hessian matrix using previous gradient information.
- **Pros:** Fast, robust, and requires no second derivatives. This is the default in R's `optim()` and `mle()`.

Nelder-Mead (Simplex)

- **How it works:** Uses no derivatives. It constructs a simplex, a geometric shape of points, that reflects and shrinks over the surface to find the peak.
- **Pros:** Extremely robust; works on non-smooth functions and poor starting values.
- **Cons:** Slow; struggles in high-dimensional problems with many parameters.

MLE Optimization in R

The Negative Log-Likelihood Trick

- R's optimization functions such as `optim()` and `nlm()` are designed to minimize, not maximize.
- To compute the Maximum Likelihood Estimate, we minimize the Negative Log-Likelihood.
- If $\ell(\theta)$ is the log-likelihood, we minimize $-\ell(\theta)$.

Using `log=TRUE`

- When computing likelihoods in R, always use `log=TRUE` inside density functions, for example `dgamma(x, shape, scale, log=TRUE)`.
- This computes the log-probability directly, which is more numerically stable than computing a very small probability and then taking its logarithm.

Optimization Pitfalls

- **Local Maxima:** The algorithm may converge to a smaller local peak instead of the global maximum.
- **Solution:** Try multiple starting values. If all runs converge to the same point, you likely found the global maximum. If not, the likelihood may be multimodal.
- **Check Convergence:** In R, `optim()$convergence == 0` indicates successful convergence. Any non-zero value indicates failure.

Why We Love MLE (Theoretical Properties)

Even when computed numerically, MLE has excellent theoretical properties.

1. **Consistency:** As sample size $n \to \infty$, $\hat{\theta} \to \theta$.
2. **Equivariance:** If $\hat{\theta}$ is the MLE of θ, then $g(\hat{\theta})$ is the MLE of $g(\theta)$.
3. **Asymptotic Normality:** For large samples, $\hat{\theta} \approx \mathcal{N}\left(\theta, \frac{1}{I(\theta)}\right)$, where $I(\theta)$ is the Fisher Information.
4. **Asymptotic Efficiency:** For large samples, the MLE achieves the minimum possible variance among regular estimators.

The Likelihood Ratio Test (LRT)

Concept

Used to compare two nested models to determine whether additional parameters significantly improve model fit.

- H_0 (Restricted Model): Parameters are fixed, for example a fair coin with $p = 0.5$.
- H_1 (Unrestricted Model): Parameters are estimated using MLE, for example $p = \hat{p}$.

The Test Statistic

$$\Lambda = -2\left[\ell(\hat{\theta}_0) - \ell(\hat{\theta})\right]$$

- $\ell(\hat{\theta})$: Log-likelihood of the unrestricted model.
- $\ell(\hat{\theta}_0)$: Log-likelihood of the restricted model.

The Distribution

- Under H_0, $\Lambda \sim \chi^2_{df}$
- Degrees of freedom df equal the number of restrictions imposed under H_0.

Profile Likelihood & Confidence Intervals

- Since the LRT statistic follows a χ^2 distribution asymptotically, we can invert the test to construct confidence intervals without assuming normality.
- In R, `confint(fit)` computes profile likelihood confidence intervals.

Week 5: Generalised Linear Models

Linear Regression Review and Diagnostics

- Unlike machine learning which focuses on prediction, this module focuses on inference to understand relationships, quantify uncertainty, and compare competing models.
- **Linear Model:**

$$y = \beta_0 + \beta_1 x + \epsilon$$

  where $\epsilon \sim N(0, \sigma^2)$.

- Interpreting the summary output:
  - The p-value tests the null hypothesis $H_0 : \beta = 0$.
  - While often informally described as the probability that the coefficient occurred by chance, it precisely measures the probability of observing an estimate this far from zero if the true coefficient were actually zero.
  - The F-statistic tests whether all slope coefficients are simultaneously zero ($H_0 : \beta_1 = \cdots = \beta_p = 0$).
- Diagnostic plots are critical for checking assumptions:
  - **Residuals vs Fitted:** Looks for non-linear patterns; should display random scatter.
  - **Normal Q-Q:** Checks if errors are normally distributed; points should follow the diagonal line.
  - **Scale-Location:** Checks for constant variance (homoscedasticity); a funnel shape indicates a violation.
  - **Residuals vs Leverage:** Identifies highly influential points pulling the regression line.

Limitations of Linear Regression

- Linear regression assumes the response Y is continuous and unbounded.
- It assumes a direct linear relationship between predictors and the mean response.
- It assumes errors are normally distributed with constant variance.
- Forcing binary data (pass/fail), count data, or positive right-skewed data into a linear model leads to invalid predictions and violated assumptions.

Introduction to Generalised Linear Models (GLMs)

- Introduced by Nelder and Wedderburn (1972), GLMs unify various regression models under one framework.
- A GLM consists of three core components:
  - **Random Component:** Specifies that the response Y_i follows a distribution from the exponential family (e.g., Normal, Binomial, Poisson, Gamma).
  - **Systematic Component:** The linear predictor combining the predictors.

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$$

  - **Link Function:** Connects the mean μ_i to the linear predictor η_i.

$$g(\mu_i) = \eta_i$$

Common GLM Families

- **Normal:** Uses the Identity link ($g(\mu) = \mu$) for continuous, roughly symmetric data.
- **Binomial:** Uses the Logit link for binary responses or proportions.
- **Poisson:** Uses the Log link ($g(\mu) = \log \mu$) for count data.
- **Gamma:** Uses the Log link for positive, right-skewed continuous data.

Logistic Regression

- Used when the response Y is binary, modelling the probability of success $p = P(Y = 1)$.
- It applies the logit (log-odds) transformation to ensure predictions remain within the valid $[0, 1]$ bounds.
  - **Logit Link:**

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Note on Probability and Odds Conversions: Converting between probability and odds is essential for interpretation.

- **Odds Formula:**

$$\text{odds} = \frac{p}{1-p}$$

- **Probability Formula:**

$$p = \frac{\text{odds}}{1 + \text{odds}}$$

Interpreting Coefficients

- Logistic regression coefficients are interpreted as Odds Ratios.
  - **Odds Ratio:**

$$e^{\beta_j}$$

- If $e^{\beta_j} > 1$: Increasing x_j by 1 unit increases the odds of success.
- If $e^{\beta_j} < 1$: Increasing x_j by 1 unit decreases the odds of success.
- If $e^{\beta_j} = 1$ ($\beta_j = 0$): The predictor has no effect.
- To compute confidence intervals directly on the odds ratio scale in R, use `exp(confint(model))`.

Nested Models and Interactions

- **Likelihood Ratio Test (LRT):**
  - Used to formally compare a reduced model (H_0) against a full model (H_1) to determine if adding predictors significantly improves the fit.
  - In R, this is executed using the `anova()` function with a Chi-Squared test. `anova(model1, model2, test="Chisq")`
- **Interactions and R Formula Syntax (`+`, `*`, `:`):**
  - An interaction implies that the effect of one predictor depends on the level of another predictor.
  - **`+` (Plus):** Adds independent main effects to the model without an interaction (e.g., `y ~ x1 + x2`).
  - **`*` (Asterisk):** Specifies an interaction *and* automatically includes the corresponding main effects (e.g., `y ~ x1 * x2` is identical to `y ~ x1 + x2 + x1:x2`).
  - **`:` (Colon):** Specifies *only* the interaction term between predictors without adding the main effects.

Fitting GLMs

- GLMs are fitted using Maximum Likelihood Estimation (MLE).
- The objective is to find the parameter values $\hat{\beta}$ that minimize the negative log-likelihood.
- **Iteratively Reweighted Least Squares (IRLS):**
  - This is the specific numerical algorithm used to optimize the log-likelihood for GLMs.
  - R's `glm()` function uses IRLS, which converges more accurately and efficiently for these specific models than general-purpose optimizers like `mle()`.

Week 6: Generalised Linear Models (Continued)

Model Evaluation and Deviance

- Understanding Maximum Likelihood Estimation (MLE) provides a principled framework for comparing models, deriving standard errors, and evaluating overall model fit.
- **Model Deviance:**

$$D = -2[l(\hat{\beta}) - l(\text{saturated model})]$$

- A "saturated" model fits perfectly by using n parameters for n observations; deviance measures how much worse the fitted model is in comparison.
- For Generalised Linear Models (GLMs), deviance plays the same role that the Residual Sum of Squares (RSS) plays in standard linear regression.
- **Residual Deviance:** Represents the amount of unexplained variation remaining after fitting the model.
- If a model fits well, the residual deviance should be approximately equal to the residual degrees of freedom ($n - p$, where p is the number of estimated parameters).

Model Comparison Tools

Likelihood Ratio Test (LRT)

The LRT formula ($\Lambda = -2[\ell(\hat{\theta}_0) - \ell(\hat{\theta})]$) and its χ^2 distribution were introduced in Week 4. Its application to GLMs via `anova()` was covered in Week 5. See those sections for the full details.

- In the GLM context, LRT compares two **nested models** where the reduced model is a special case of the full model. A large Λ (small p-value) indicates the extra parameters significantly improve the fit.

Information Criteria (AIC and BIC)

- Used for comparing **non-nested models** or for automated model selection. Both criteria penalise adding extra parameters to prevent overfitting.
- **Akaike Information Criterion (AIC):**

$$AIC = -2l(\hat{\beta}) + 2k$$

- **Bayesian Information Criterion (BIC):**

$$BIC = -2l(\hat{\beta}) + k \log n$$

- For both metrics, a lower value indicates a better model. BIC applies a stronger penalty for complexity in large datasets, thereby favouring simpler models.

Poisson Regression for Count Data

- Designed for response variables that are non-negative integer counts (e.g., number of insurance claims, hospital admissions).
- Applying standard linear regression to count data causes invalid predictions (negative counts), violates normality assumptions, and fails to capture that variance increases with the mean.
- **Poisson Distribution Properties:** The expected mean and variance are identical ($E(Y) = \lambda$, $Var(Y) = \lambda$).
- The Poisson GLM connects the log of the mean to the linear predictors.
- **Systematic Component:**

$$\log(\lambda_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$$

Offsets

- Used when modelling a rate (e.g., claims per policyholder) rather than a raw total count across groups of differing sizes.
- Using `offset(log(exposure))` in the model formula constrains the coefficient of the exposure variable to exactly 1.
- **Offset Formula:**

$$\log\left(\frac{\lambda_i}{Exposure_i}\right) = \beta_0 + \beta_1 x_{i1} + ...$$

Handling Categorical Predictors in R

- **Dummy Coding (N-1 Bits):** When dealing with categorical data, avoid standard "one-hot encoding" (e.g., representing 4 districts as 1000, 0100, 0010, 0001). Instead, R uses an $n-1$ bit representation to avoid multicollinearity. For a 4-level variable, level 1 acts as the baseline (000), and three indicator variables represent the rest (100, 010, 001).
- **Orthogonal Polynomials:** For ordered factors, R automatically uses orthogonal polynomial contrasts (`.L` for linear, `.Q` for quadratic, `.C` for cubic trends). If standard dummy coding is preferred, it must be explicitly forced using `contr.treatment`.
- **Formula Implementation:** Ensure categorical data is explicitly cast before modelling to trigger correct dummy coding (e.g., wrapping variables in `as.factor()` directly within the formula or dataset).

Variance Functions and Overdispersion

- In standard linear regression, variance is assumed constant. In GLMs, variance is a function of the mean.
- **Variance Function:**

$$Var(Y_i) = \phi \cdot V(\mu_i)$$

- For the Poisson and Binomial families, the dispersion parameter ϕ is theoretically fixed at 1.
- **Overdispersion:** Occurs frequently in real-world data when the observed variance is strictly greater than the mean ($Var(Y) > \mu$).
- Ignoring overdispersion results in standard errors that are too small, p-values that trigger false positives, and artificially narrow confidence intervals.

Checking for Overdispersion

- **Dispersion Statistic ($\hat{\phi}$):**

$$\hat{\phi} = \frac{\text{Residual Deviance}}{\text{Residual df}}$$

- **Evaluation Rules of Thumb:**
  - $\hat{\phi} \approx 1$: The model fits the variance assumption perfectly.
  - $\hat{\phi} \approx 2$: The fit is questionable; exploration of alternative models is recommended.
  - $\hat{\phi} \geq 3$: Hard cut-off. The Poisson model is drastically overconfident, its p-values cannot be trusted, and it must be discarded in favour of an overdispersion-tolerant model.

Dealing with Overdispersion

Quasi-Poisson Model

- Relaxes the rigid assumption by estimating the dispersion parameter ϕ directly from the data.
- The resulting coefficients remain completely identical to the standard Poisson model, but the standard errors are widened by multiplying them by $\sqrt{\hat{\phi}}$.
- Because it relies on quasi-likelihood rather than true maximum likelihood, no AIC value is generated for model comparison.

Negative Binomial Regression

- A fully parametric alternative specifically designed for overdispersed count data.
- Introduces a dedicated dispersion parameter r.

- **Negative Binomial Variance:**

$$Var(Y) = \mu + \frac{\mu^2}{r}$$

- Unlike Quasi-Poisson, it provides a proper log-likelihood, enabling the use of AIC for direct model comparison.
- Implemented in R using `glm.nb()` from the `MASS` package.

Positive Continuous Responses (Gamma GLM)

- Used when the response variable is strictly positive ($Y > 0$), exhibits a right-skewed distribution, and possesses a variance that increases with the mean (e.g., claim financial amounts, hospital lengths of stay).
- **Gamma Variance:** Grows quadratically with the mean (μ^2/α).
- **Link Function:** While the canonical link for the Gamma distribution is the inverse ($1/\mu$), the **log link** ($\log(\mu)$) is almost always preferred in practice.
- The log link guarantees strictly positive predictions and preserves a clean, multiplicative interpretation of the coefficients (identical to Poisson interpretation).
- Must be explicitly specified in R: `family = Gamma(link = "log")`.

Quick Reference: Coefficient Interpretation

- For log-link models, coefficients must be exponentiated to evaluate their effect on the interpretable scale.

Model	Link Function	Interpretation of Exponentiated Coefficient ($e^{\hat{\beta}_j}$)
Logistic	Logit	**Odds ratio:** The odds of $Y = 1$ multiply by $e^{\hat{\beta}_j}$ per 1-unit increase in X_j.
Poisson	Log	**Rate ratio:** The expected count multiplies by $e^{\hat{\beta}_j}$ per 1-unit increase in X_j.
Gamma	Log	**Mean ratio:** The expected continuous positive response multiplies by $e^{\hat{\beta}_j}$.
Linear	Identity	**Additive effect:** The expected response increases directly by the raw coefficient $\hat{\beta}_j$.

Week 7: Density Estimation

Motivation and Introduction to Density Estimation

- In previous weeks, we relied on known mathematical density functions (e.g., Normal, Gamma, Chi-Squared) to describe data, replacing parameters like μ and σ^2 with sample estimates.
- This standard approach is called the parametric approach.
- However, real-world sample data often does not resemble any known standard distribution, making the parametric approach invalid.
- Density estimation aims to approximate the probability density function (pdf) of a random variable directly from the data, allowing the data to "speak for themselves" through non-parametric procedures.

Histograms and Local Averages

- Histograms act as a basic form of density estimation by counting and plotting the number of observations in defined bins.
- Histograms have limitations: the bins must be defined, and the resulting shape can be very blocky depending on the chosen bin width.
- We can formally estimate the density $f_X(x)$ using a local average of points falling within a small window δx.
- **Local Average Estimate:**

$$\hat{f}_X(x) = \frac{1}{n\delta x} \sum_{i=1}^{n} I(|x - x_i| < \delta x/2)$$

- Because the local average uses a strict rectangular function $I(\cdot)$, the resulting plot remains quite blocky.

Kernel Density Estimation (KDE)

- To fix the blocky nature of local averages, we replace the rectangular indicator function $I(\cdot)$ with a tapering kernel function $K(\cdot)$, producing a locally weighted average.
- **Kernel Density Estimator:**

$$\hat{f}_X(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

- In this formula, h acts as the width of the window and is known as the bandwidth.
- A valid kernel density estimate must be a true density (non-negative and integrating to 1), which is guaranteed if the chosen kernel K is itself a valid probability density function.
- **PDF Properties:** A valid density must satisfy $f(x) \geq 0$ and $\int f(x)\,dx = 1$.
- **KDE Interpretation:** KDE can be viewed as a sum of small "bumps" (kernels) centred at each data point; the final estimate is the average of these contributions.

Choice of Kernel

- **Rectangular Kernel:** Corresponds to the basic moving average ($K(t) = 1/2$ for $|t| < 1$).
- **Triangular Kernel:** Produces a less blocky result than the rectangular kernel.
- **Normal (Gaussian) Kernel:** A highly popular choice based on the standard normal distribution.
- **Epanechnikov Kernel:** Theoretically the most efficient kernel.
- Efficiency refers to how accurately the kernel estimates the true underlying density while minimizing variance; all the above kernels have high efficiency (greater than 90%).

Choice of Bandwidth

- The bandwidth h controls the smoothness of the estimate; too small creates an "undersmoothed" jagged line, while too large creates an "oversmoothed" flat line.

Visual Effects of Bandwidth

Bandwidth	Appearance	Problem	Use Case
Very Small (h « optimal)	Jagged, spiky, many peaks	Overfitting to noise; hard to see true structure	Rarely intentional
Small (h < optimal)	Somewhat detailed, shows structure but bumpy	Minor noise visible	Exploratory analysis
Optimal	Smooth curve, shows true peaks/valleys clearly	None	**Preferred for inference**
Large (h > optimal)	Very smooth, rounded	Hides true features; merges separate modes	Quick visualization only
Very Large (h » optimal)	Nearly flat, almost no variation	Loses all information	Not useful

Visual Interpretation Tips

- **If plot is spiky with many small peaks:** bandwidth is too small → increase it
- **If you see clear bimodality (two peaks):** bandwidth is reasonable (or slightly large)
- **If plot is almost flat:** bandwidth is too large → decrease it

- **If you can't tell if there are 1 or 2 modes:** try 3-4 bandwidths and compare

Selecting Bandwidth

- **Silverman's Rule of Thumb:**

$$h \approx 1.06\sigma n^{-1/5}$$

- The rule of thumb assumes the true underlying distribution is Normal, and R uses a pragmatic variant of this ($h = 0.9 \min(s, R/1.34) n^{-1/5}$) as its default `bw="nrd0"`.
- **Cross-Validation:** An alternative **data-driven** method that minimizes the integrated square error by leaving out one observation at a time.
  - **Unbiased cross-validation:** `bw.ucv()` - often selects **smaller** bandwidth than rule of thumb
  - **Biased cross-validation:** `bw.bcv()` - balance between fit quality and smoothness
  - Cross-validation is preferred when you want bandwidth selected **automatically from data**
- **Cross-Validation Formula:** Bandwidth can be chosen by minimising:

$$M(h) = \int \hat{f}(x)^2 \, dx - \frac{2}{n} \sum_{i=1}^{n} \hat{f}_{-i}(x_i)$$

Comparing Bandwidths Visually (Exam Strategy) When presented with multiple KDE plots at different bandwidths:

1. **Look for clear structure:** Which plot shows distinct modes/peaks most clearly?
2. **Check for noise artifacts:** Are there unexplained spikes or bumps?
3. **Consider data properties:** If you suspect 1-2 modes, choose the bandwidth that reveals this
4. **Balance:** Too much smoothing loses information; too little adds noise
5. **Answer the scientific question:** If you're looking for subpopulations, a slightly smaller bandwidth may be better

Exam Tip: For Q4(b)(ii) style questions, justify by saying: - "This bandwidth shows the underlying structure clearly (bimodal/unimodal)" - "Smaller bandwidths are too noisy; larger ones over-smooth the important features" - "This bandwidth achieves the best balance between revealing structure and suppressing noise"

Edge Effects

- KDE performs poorly near boundaries (e.g. data ≥ 0).
- Can be improved using transformations (e.g. log transform).

Special Data Types

- **Periodic data** (e.g. angles): require periodic kernels.
- **Positive-only data:** often handled via transformations.

Computational Complexity

- A naive implementation of KDE evaluates the kernel at m grid points for every n observation, resulting in $O(nm)$ operations, which is very slow for large datasets.
- KDE is fundamentally a convolution of the kernel and a binned histogram.
- By using the Fast Fourier Transform (FFT), convolutions can be computed in $O(m \log m)$ operations, making it independent of the sample size n.
- R's built-in `density()` function utilizes this FFT method internally for rapid performance.

Applications (Bayesian Classification)

- Beyond displaying data, density estimation can be used to construct non-parametric classifiers.
- Using Bayes' theorem, we can classify the probability of a condition C given a test measurement X.
- **Posterior Probability (Bayes Classifier):**

$$P(C|X) = \frac{f(X|C)P(C)}{f(X)}$$

- The densities $f(X|C)$ (e.g., blood pressure given diseased vs. healthy) can be estimated directly using KDE.
- **Mixture vs. Marginal Denominator:** The denominator $f(X)$ can be calculated in two ways.
  - *Mixture:* Theoretically correct, built from the conditional densities using the Law of Total Probability: $pf_1(x) + (1-p)f_0(x)$.
  - *Marginal:* The density estimated directly from all combined patient data.
- **Mixture vs. Marginal Insight:** Theoretically equivalent, but in practice they differ slightly due to smoothing/estimation error.

Multivariate Densities

- Density estimation scales to higher dimensions; for 2D data, a two-dimensional kernel is placed at each data point, and the results are averaged.
- In R, 2D density estimation is handled by the `kde2d` function from the `MASS` package.
- By default, `kde2d` selects bandwidths for each dimension independently using the rule of thumb; more principled matrix-valued bandwidth selection is available via the `ks` package.
- **Multivariate Bandwidth Detail:** Separate bandwidths for each dimension provide basic flexibility; a full bandwidth matrix (e.g. the `ks` package) allows for correlation-aware smoothing.

R Functions to Know

- `density()` – uses FFT internally
- `bw.nrd0` – rule of thumb
- `bw.ucv`, `bw.bcv` – cross-validation
- `geom_density()` – ggplot
- `kde2d()` – 2D KDE in `MASS`

Collinearity Review

- Collinearity occurs when there is a strong correlation between two predictor variables, violating the desire for independent explanatory variables.
- High collinearity leads to unstable coefficients, difficult interpretation, and high variance in standard errors (making p-values unreliable).

- When creating dummy variables for categorical data, including all categories creates "perfect multicollinearity" because one category can be perfectly predicted by the absence of the others.
- The solution is to always exclude one level to serve as the baseline/reference category.

Week 8: Mixtures

Motivation

- Distributions can often be a mixture of underlying sub-populations (e.g., commute times by different travel modes).
- Unlike Kernel Density Estimation (KDE), which places an identical kernel at every data point, a mixture model uses a smaller number of components.
- Components need not have the same weight, need not be centred at a data point, and need not have the same standard deviation.

Mixture Distributions

- Suppose there are d underlying distributions with probability density functions $f_1(x), \ldots, f_d(x)$.
- An observation is drawn from f_j with probability p_j (the mixing weight), where $p_j > 0$ and the sum of all mixing weights equals 1.
- **Mixture Density:**

$$f(x) = \sum_{j=1}^{d} p_j f_j(x)$$

Two-Component Normal Mixture

- For a two-component Normal mixture, the density is defined by the components and a mixing parameter $\lambda \in (0, 1)$.
- **Two-Component Density:**

$$f(x) = \lambda f_1(x) + (1 - \lambda) f_2(x)$$

- This model requires estimating exactly five parameters: μ_1, σ_1, μ_2, σ_2, and the mixing parameter λ.

Estimating the Parameters

- For a single Normal distribution, Maximum Likelihood Estimation (MLE) provides simple closed-form estimates.
- For mixtures, the log-likelihood function does not simplify easily, and direct numerical maximization is unreliable and sensitive to starting values.
- **The Incomplete Data Problem:** The difficulty arises because we do not know the component label z_i (which specific distribution generated the observation x_i).

The Expectation-Maximization (EM) Algorithm

- If the component labels z_i were known, estimation would be trivial.
- The EM algorithm solves the incomplete data problem by iterating between two steps:
  - **Expectation Step:** Calculates the expected value (probability) of the missing component labels z_i given the current parameter estimates.
  - **Maximization Step:** Updates the parameter estimates (μ, σ, and mixing weights) using these expected labels.
- The algorithm repeats these two steps continuously until the parameters converge to a stable solution.

EM Algorithm: Key Formulas

- Define the **responsibility** (expected value of z_i):

$$\gamma_i = \frac{\lambda f_1(x_i)}{\lambda f_1(x_i) + (1-\lambda) f_2(x_i)}$$

- These γ_i represent the probability that observation x_i belongs to component 1.
- **Weighted parameter updates (M-step):**

$$\mu_1 = \frac{\sum \gamma_i x_i}{\sum \gamma_i}, \quad \sigma_1^2 = \frac{\sum \gamma_i (x_i - \mu_1)^2}{\sum \gamma_i}$$

$$\mu_2 = \frac{\sum (1-\gamma_i) x_i}{\sum (1-\gamma_i)}, \quad \sigma_2^2 = \frac{\sum (1-\gamma_i)(x_i - \mu_2)^2}{\sum (1-\gamma_i)}$$

$$\lambda = \frac{\sum \gamma_i}{n}$$

- Each observation contributes fractionally to each component via γ_i.

Properties of EM Algorithm

- The likelihood **increases at every iteration**.
- Converges to a **local maximum** (not necessarily global).
- Sensitive to **initial parameter values** $\rightarrow$ multiple initialisations are often used.
- More stable than direct numerical maximization.

Extension to k Components

- For k components, we define γ_{ij} = probability that observation i belongs to component j.
- For each observation:

$$\sum_{j=1}^{k} \gamma_{ij} = 1$$

- EM generalises by updating all component parameters using weighted sums.

Number of Components

- Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) apply to mixture models to help select the optimal number of components k.

AIC and BIC

- **AIC:**

$$AIC = -2\log\mathcal{L} + 2p$$

- **BIC:**

$$BIC = -2\log\mathcal{L} + \log(n)\,p$$

- BIC penalises model complexity more heavily than AIC.
- **Parameter Count Example:** For a k-component mixture of bivariate Normals, each component has 2 means and 3 unique covariance parameters (5 parameters per component), plus $k-1$ free mixing weights.
- **Total Parameters (p):**

$$p = 5k + (k-1) = 6k - 1$$

Mixture of Regressions

- Instead of mixing distributions, we mix regression models.
- Each observation is generated from one of several regression lines:

$$y \sim \begin{cases} \mathcal{N}(\alpha_1 + \beta_1 x, \sigma^2) & \text{with probability } \lambda \\ \mathcal{N}(\alpha_2 + \beta_2 x, \sigma^2) & \text{with probability } 1-\lambda \end{cases}$$

- EM algorithm applies:
  - E-step: compute probabilities of belonging to each regression
  - M-step: fit **weighted regressions**

Multivariate Mixtures

- Observations can be vectors instead of scalars.
- Each component is a **multivariate Normal distribution** with:
  - Mean vector
  - Covariance matrix
- EM generalises by:
  - Computing γ_{ij} for each component
  - Updating mean vectors and covariance matrices using weighted sums

---

Week 9: Markov Chains

Motivation: Beyond Independence

- Standard statistical modeling often assumes that observations are independent events or measurements.
- When measurements are sequential and dependent (e.g., forecasting whether today will be Sunny, Cloudy, or Rainy based on previous days), independence models are too naive.
- A sequence of dependent observations can be effectively modeled as a Markov Chain.

The Markov Property

- A system satisfies the Markov Property if the future state depends strictly on the current state, and not on the sequence of historical events that preceded it.
- **The Markov Property:**

$$P(X_t \mid X_{t-1}, X_{t-2}, \dots, X_0) = P(X_t \mid X_{t-1})$$

- We focus on **Discrete-Time Finite-State Markov Chains**, where:
  - Time is discrete
  - The state space is finite

Transition Matrices

- A **Homogeneous Markov Chain** (where transition probabilities do not change over time) is fully defined by its state space and a transition matrix P.
- Transition probabilities:

$$p_{ij} = P(X_t = S_j \mid X_{t-1} = S_i)$$

- Properties:
  - $0 \leq p_{ij} \leq 1$
  - $\sum_j p_{ij} = 1$ for each i
  - Such a matrix is called a **stochastic matrix**
- Each row represents the probability distribution of the next state given the current state.

Graph Interpretation

- A Markov chain can be represented as a directed graph:
  - Nodes = states
  - Edges = transitions
  - Edge weights = transition probabilities

Homogeneous vs Non-Homogeneous

- **Homogeneous:** transition probabilities are constant over time
- **Non-homogeneous:** transition probabilities vary with time $p_{ij}(t)$

Multi-Step Transitions

- Two-step transitions:

$$P(X_t = S_j \mid X_{t-2} = S_i) = \sum_k p_{ik} p_{kj}$$

- General case:

$$P(X_t = S_j \mid X_{t-n} = S_i) = (P^n)_{ij}$$

Worked Example: Two-Step Transition Calculation Given transition matrix:

$$P = \begin{pmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.6 & 0.2 \\ 0.1 & 0.3 & 0.6 \end{pmatrix}$$

Question: Calculate $P(X_2 = C \mid X_0 = A)$ (probability of going from state A to C in 2 steps)

Solution:

Using the formula: $P(X_2 = C \mid X_0 = A) = \sum_k p_{AK} p_{KC}$

Where K can be A, B, or C:

$$P(X_2 = C \mid X_0 = A) = p_{AA} \cdot p_{AC} + p_{AB} \cdot p_{BC} + p_{AC} \cdot p_{CC}$$

$$= (0.5)(0.1) + (0.4)(0.2) + (0.1)(0.6)$$

$$= 0.05 + 0.08 + 0.06 = 0.19$$

Interpretation: The probability of transitioning from state A to state C in exactly 2 steps is 0.19 or 19%.

Alternative (Matrix Method):

Compute $P^2 = P \times P$:

$$(P^2)_{AC} = (\text{Row A of P}) \cdot (\text{Column C of P}) = 0.19$$

Evolution of State Distribution

- If the initial distribution is p_0, then after n steps:

$$p_n = p_0 P^n$$

Structure of Markov Chains

Accessibility

- State j is **accessible** from state i if there exists $n \geq 1$ such that:

$$(P^n)_{ij} > 0$$

Irreducibility

- A chain is **irreducible** if every state is accessible from every other state.

Communicating Classes

- States that can reach each other form a **communicating class**.
- A class is **closed** if it cannot be left once entered.

Absorbing States

- A state i is **absorbing** if:

$$p_{ii} = 1$$

- Once entered, it cannot be left.

Periodicity

- A state has period d if it can only return to itself in multiples of d steps.
- If $d = 1$, the state is **aperiodic**.
- A chain is aperiodic if all states are aperiodic.

Ergodicity

- A chain is **ergodic** if it is:
  - Irreducible
  - Aperiodic

Stationary Distribution

- A probability vector π is a **stationary distribution** if:

$$\pi = \pi P$$

- Properties:
  - $\sum_i \pi_i = 1$
  - $\pi_i \geq 0$
- Interpretation:
  - If the chain starts in π, it remains in π forever
  - Represents long-run proportions of time spent in each state

Ergodic Theorem

For an ergodic chain:

1. A **unique stationary distribution** π exists
2. Convergence:

$$p_n \to \pi \quad \text{as } n \to \infty$$

3. Long-run averages converge:

$$\frac{1}{n} \sum_{t=1}^{n} f(X_t) \to \sum_i \pi_i f(S_i)$$

- Convergence happens regardless of the initial distribution.

Computing the Stationary Distribution

Power Method

- Start with any probability vector x
- Iteratively compute:

$$x \leftarrow xP$$

- Repeat until convergence

Eigenvalue Method

- Solve:

$$\pi P = \pi$$

- Equivalent to finding the eigenvector corresponding to eigenvalue 1

First Passage Time and Expected Passage Time

- **First Passage Time (T_{ij}):** Number of steps required to reach state j for the first time starting from state i
- Let $h_{ij}(n) = P(T_{ij} = n)$:

$$h_{ij}(1) = p_{ij}$$

$$h_{ij}(n) = \sum_{k \neq j} p_{ik}\, h_{kj}(n-1)$$

- **Expected Passage Time:**

$$E[T_{ij}] = \sum_{n} n \cdot h_{ij}(n)$$

Mean Return Time

- Mean return time to state i:

$$\mu_i = E[T_{ii}]$$

- Key relationship:

$$\pi_i = \frac{1}{\mu_i}$$

- States with higher stationary probability are visited more frequently and have shorter return times.

Maximum Likelihood Estimation (MLE) of Markov Chains

- We can estimate transition probabilities from observed data.
- Let n_{ij} = number of observed transitions from state i to state j
- **MLE:**

$$\hat{p}_{ij} = \frac{n_{ij}}{\sum_j n_{ij}}$$

- Interpretation:
  - Count transitions
  - Normalize each row

Laplace Smoothing

- To handle unseen transitions:

$$\hat{p}_{ij} = \frac{n_{ij} + \alpha}{\sum_j (n_{ij} + \alpha)}$$

- Prevents zero probabilities

R Implementation (markovchain package)

- **Defining a Markov Chain:**

```
library(markovchain)

P <- matrix(c(0.5, 0.3, 0.2,
              0.2, 0.6, 0.2,
              0.15, 0.05, 0.8),
            byrow=TRUE, nrow=3)

mc <- new("markovchain",
          transitionMatrix = P,
          states = c("Cloudy", "Rainy", "Sunny"))
```

- **Multi-step transitions:**

```
p0 <- c(0, 0, 1)
p0 * mc        # one step
p0 * mc^5      # five steps
```

- **Stationary distribution and return times:**

```
steadyStates(mc)
meanRecurrenceTime(mc)
```

- **Simulation:**

```
markovchainSequence(30, mc, t0="Sunny")
```

- **Fitting from data:**

```
markovchainFit(data)$estimate
```

Applications of Markov Chains

- Web search (e.g., PageRank algorithm)
- Finance (credit rating transitions, regime-switching models)
- Genomics (DNA sequence modeling)
- Natural language processing (text generation, predicting the next word)

Week 10: Bayesian Inference

Motivation: Frequentist vs Bayesian

- **Frequentist Approach (e.g., Maximum Likelihood):** Treats the parameter θ as a fixed, unknown quantity. The goal is to find the estimate $\widehat{\theta}$ that makes the observed data most probable.
- **Bayesian Approach:** Treats θ as a random variable with its own probability distribution. The goal is to answer: *Given the data I have observed, what should I now believe about θ?*
- Bayesian inference is closely related to earlier uses of Bayes' theorem (e.g. medical testing):
  - Prior = initial belief (e.g. disease prevalence)
  - Likelihood = evidence (e.g. test result accuracy)
  - Posterior = updated belief after observing data

Bayes Theorem for Inference

- Bayesian inference is entirely built upon Bayes' Theorem, which updates our prior beliefs based on new evidence.
- **Bayes Theorem:**

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

- The components of the theorem:
  - **Prior ($p(\theta)$):** What you believe about θ before seeing the data.
  - **Likelihood ($p(y \mid \theta)$):** How probable the observed data is for a specific value of θ.
  - **Posterior ($p(\theta \mid y)$):** What you should believe about θ after seeing the data.
  - **Marginal Likelihood / Evidence ($p(y)$):** A normalizing constant to ensure the posterior is a valid probability distribution that integrates to 1.
- The marginal likelihood is defined as:

$$p(y) = \int p(y \mid \theta)p(\theta)\, d\theta$$

- Because the denominator $p(y)$ is just a constant for a given dataset, the relationship is often simplified to:
- **Proportionality Rule:**

$$p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$$

Conjugacy and Conjugate Priors

- Calculating the normalizing constant $p(y)$ often requires extremely complex integration.
- A **Conjugate Prior** is a prior distribution chosen specifically so that the resulting Posterior distribution belongs to the exactly same probability family.

- This provides a simple, closed-form algebraic solution, bypassing the need for numerical integration entirely.

The Beta-Binomial Model

- Used for modeling proportions or probabilities of success (e.g., defect rates, coin flips).
- **Likelihood:** The observed data y follows a Binomial distribution for n trials.

$$p(y \mid \theta) = \binom{n}{y} \theta^y (1-\theta)^{n-y}$$

- **MLE (for comparison):**

$$\hat{\theta}_{MLE} = \frac{y}{n}$$

- **Prior:** The parameter θ follows a Beta distribution.
  - The Beta distribution is naturally bounded between 0 and 1, making it the perfect choice for modeling probabilities.
  - It is defined by two hyperparameters: α (prior successes) and β (prior failures).
- **Uniform Prior (special case):**
  - $p(\theta) = 1$ for $0 \leq \theta \leq 1$
  - Equivalent to $\text{Beta}(1, 1)$
  - Interpretation:
    - No preference for any value of θ
    - Often called a **non-informative prior**
    - Posterior is driven entirely by the data
- **Interpretation of Beta(α, β):**
  - $\alpha - 1$ = prior "successes"
  - $\beta - 1$ = prior "failures"
  - Total prior strength = $\alpha + \beta - 2$ (pseudo-observations)
- **Posterior:** Because the Beta distribution is conjugate to the Binomial distribution, the posterior is guaranteed to be another Beta distribution.
- **Posterior Distribution Formula:**

$$\theta \mid y \sim \text{Beta}(\alpha + y, \beta + n - y)$$

Prior - Likelihood - Posterior Relationship

- **Prior:** belief before seeing data
- **Likelihood:** evidence from observed data

- **Posterior:** updated belief after combining both
- The posterior is a **compromise between prior and data**:
  - Small dataset → prior has strong influence
  - Large dataset → likelihood dominates
  - As n increases → posterior concentrates around $\frac{y}{n}$
- Larger $\alpha + \beta \Rightarrow$ **stronger prior** (less influenced by data)

Example: Factory Defect Rate

- **Prior Belief:** We have strong historical evidence that the defect rate is exactly 1%. We model this as $\theta \sim \text{Beta}(1, 99)$.
- **New Data:** We test 10 new items and find 2 defective ($n = 10$, $y = 2$). The raw sample defect rate is 20%.
- **Posterior Update:** Using the conjugate update rule, the new distribution becomes $\text{Beta}(1+2, 99+10-2) = \text{Beta}(3, 107)$.
- **Interpretation:** The posterior shifts upward slightly but does not jump to the 20% seen in the data. Because the prior was very strong (based on essentially 100 prior observations) relative to the small new dataset ($n = 10$), the prior heavily dominates the final posterior result.

Bayesian Point Estimates and Intervals

- Instead of standard errors and frequentist p-values, Bayesian inference uses the posterior distribution directly to answer questions.
- **Maximum A Posteriori (MAP):** The mode (highest peak) of the posterior distribution. It represents the single most probable value of θ.
- **MAP vs MLE:**
  - MLE maximises likelihood
  - MAP maximises posterior
  - MAP can be seen as MLE with **regularisation from the prior**
  - With a uniform prior, MAP $\approx$ MLE
- **MAP Formula for Beta Distribution:**

$$\text{MAP} = \frac{\alpha - 1}{\alpha + \beta - 2}$$

  - Valid only when $\alpha, \beta > 1$
- **Credible Interval:** The Bayesian alternative to the Confidence Interval.
  - A 90% Credible Interval means: *"There is a 90% probability that θ lies within this range."*
  - Typically computed using posterior quantiles (e.g. 5th and 95th percentiles)

R Implementation

- In R, Bayesian updating with conjugate priors is simply a matter of updating the shape parameters of the distribution.

```
n <- 10 # Total trials
y <- 2  # Observed successes
a <- 1  # Prior alpha
b <- 99 # Prior beta

# Calculate Posterior parameters
post_a <- a + y
post_b <- b + n - y

# MLE
mle <- y / n

# Calculate Maximum A Posteriori (MAP)
map_estimate <- (post_a - 1) / (post_a + post_b - 2)

# Calculate 90% Credible Interval
cred_int <- qbeta(c(0.05, 0.95), post_a, post_b)
```

Week 11: Bayesian Inference and MCMC

The Intractability Problem

- Analytical computation of the posterior requires calculating the marginal likelihood to act as a normalizing constant.
- For most realistic, non-conjugate models, the integral required to compute this constant has no closed-form analytical solution.
- Without the marginal likelihood, we cannot normalize the posterior, making it difficult to compute credible intervals or expectations directly.
- **Marginal Likelihood:**

$$p(y) = \int p(y \mid \theta)p(\theta)d\theta$$

Grid Approximation

- Grid approximation replaces the continuous parameter space with a discrete grid of finite evaluation points.
- The unnormalized posterior is computed at each grid point by multiplying the discrete prior by the likelihood, and then normalizing the resulting discrete probability mass function.
- This method is exact on the specified grid but suffers severely from the curse of dimensionality.

- A model with many parameters becomes computationally infeasible (e.g., a 5-parameter model with 100 points each requires 100^5 evaluations).

Sampling from the Posterior

- An alternative to full evaluation is to draw a sequence of random samples from the posterior.
- From these samples, we can estimate posterior means, quantiles, and credible intervals.
- Inverse transform sampling can draw samples using a uniform distribution if the Cumulative Distribution Function (CDF) and its inverse (the quantile function) are known.
- Because the CDF of an arbitrary posterior is rarely known, advanced stochastic processes are required.
- **Posterior Expectation:**

$$E[f(\theta) \mid y] \approx \frac{1}{B} \sum_{b} f(\theta^{(b)})$$

Markov Chain Monte Carlo (MCMC)

- MCMC constructs an ergodic Markov Chain designed so that its unique stationary distribution is exactly the target posterior distribution.
- By running the chain for a sufficiently long duration, the ergodic theorem guarantees that the generated sequence will converge to draws from the posterior.
- MCMC only requires evaluating the unnormalized posterior $f(\theta) \propto p(y \mid \theta)p(\theta)$, neatly bypassing the intractable integration problem.

The Metropolis-Hastings Algorithm

- The algorithm generates an MCMC sequence using a stochastic proposal mechanism $g(\theta^* \mid \theta^{(t)})$, often a Normal distribution centered on the current value.
- At each step t, a new candidate value θ^* is proposed.
- The candidate is accepted with probability $\min(1, \alpha)$; if rejected, the chain remains at the current value for that iteration.
- This accept-reject mechanism ensures the chain satisfies detailed balance, maintaining the target posterior as the stationary distribution.
- **Acceptance Ratio:**

$$\alpha = \frac{f(\theta^*)}{f(\theta^{(t)})} \cdot \frac{g(\theta^{(t)} \mid \theta^*)}{g(\theta^* \mid \theta^{(t)})}$$

Proposal Tuning and Step Size

- The standard deviation (step size) of the proposal distribution critically affects the algorithm's efficiency.

- If the step size is too small, the chain explores slowly, exhibits high autocorrelation, and has an acceptance rate that is too high (e.g., $> 70\%$).
- If the step size is too large, the chain gets stuck because most proposals are rejected, leading to a very low acceptance rate (e.g., $< 10\%$).
- A well-tuned proposal standard deviation results in good mixing and an optimal acceptance rate between 20% and 50%.

Assessing Convergence and Diagnostics

- MCMC samples are generated sequentially and are inherently dependent (autocorrelated).

Trace Plots and Burn-in

- A trace plot visualizes the sequence of parameter values across iterations to assess mixing.
- The "burn-in" period consists of the initial transient iterations where the chain is still influenced by its starting value.
- Burn-in samples are biased and must be discarded. The chain should look like stationary, mean-reverting white noise after burn-in.

Autocorrelation and Effective Sample Size (ESS)

- The Autocorrelation Function (ACF) measures the correlation between samples separated by a specific lag.
- A fast decay in the ACF plot indicates good mixing, while slow decay indicates high memory and slow exploration.
- The Effective Sample Size (ESS) estimates the equivalent number of independent samples within the autocorrelated chain. Low ESS requires running the chain longer.

Multiple Chains and Gelman-Rubin Diagnostic

- Running multiple chains from overdispersed starting values provides robust evidence of convergence if all chains mix into the identical distribution.
- The Gelman-Rubin statistic ($\widehat{R}$) formally compares within-chain variance to between-chain variance.
- $\widehat{R} \approx 1$ indicates strong agreement between chains (convergence), whereas $\widehat{R} > 1.1$ indicates failure to converge.

Worked Example: Interpreting MCMC Diagnostics **Scenario:** You run MCMC to estimate a parameter and get the following diagnostics:

Diagnostic	Value	Interpretation
Trace plot appearance	Looks like "hairy caterpillar" (mean-reverting fluctuations)	Good mixing
Trace plot appearance	Trends upward or downward; doesn't revert to mean	Poor mixing; needs more iterations
ACF decay	Drops below 0.1 at lag ~5-10	Good mixing
ACF decay	High correlation at lag 50+	Slow mixing; high autocorrelation

Diagnostic	Value	Interpretation
$\hat{R}$ value	1.01	Converged (< 1.05 benchmark)
$\hat{R}$ value	1.15	Not converged (> 1.05); run longer
ESS	4000 out of 5000 iterations	Good efficiency
ESS	100 out of 5000 iterations	Slow mixing; run much longer

Decision Rules: * If trace plots look like white noise, ACF drops quickly, and $\hat{R} < 1.05 \rightarrow$ **Use all chains** for inference * If trace plots trend or ACF is slow $\rightarrow$ **Run longer** or **tune proposal distribution** * If $\hat{R} > 1.1 \rightarrow$ **Restart completely** with better starting values or model specification

Applied Bayesian Workflow

- The transition from dataset to posterior requires an explicit, step-by-step workflow.

Step 0: Choose a Model

- Match the likelihood model to the data type (e.g., Poisson for unbounded counts, Bernoulli for binary outcomes).
- Always check model assumptions. For example, a Poisson model assumes the variance is approximately equal to the mean. If variance heavily exceeds the mean (overdispersion), a Negative Binomial model is required.

Step 1: Specify the Prior

- Select a prior distribution that reflects existing knowledge before seeing the data (e.g., a Gamma prior for a Poisson rate parameter).
- Perform a **Prior Predictive Check** by simulating data from the prior to ensure it covers the observed data range reasonably without generating physically impossible values.

Step 2: Write the Log-Posterior

- Always compute the posterior on the log scale to prevent numerical underflow in software.
- The log-posterior is the sum of the log-likelihood and the log-prior. Evaluate it at sensible bounds to ensure plausible values return higher log-probabilities than implausible ones.
- **Gamma-Poisson Log-Posterior Form:**

$$\log p(\lambda \mid y) \propto (\alpha + \sum y_i - 1) \log \lambda - (\beta + n)\lambda$$

Step 3: Visualise Before Sampling

- Plot the unnormalized posterior surface across a grid of plausible values.
- This reveals whether the posterior is multimodal or heavily skewed and confirms where the sampler should ideally be spending its time.

Step 4: Run the Sampler

- Initialize the chain deliberately off-center from the expected posterior mean.
- If the chain successfully finds the right region despite a poor start, it provides confidence that there are no major mixing problems.

Step 5 & 6: Burn-In and Post-Diagnostics

- Discard the initial iterations (e.g., 10% to 20% of the total chain) as burn-in.
- Generate trace plots and ACF plots exclusively on the post-burn-in samples to verify rapid mixing and stationary behavior.

Step 7 & 8: Posterior Summaries and Interpretation

- Extract point estimates (Posterior Mean, Posterior Median) and measures of uncertainty (95% Credible Interval) from the samples.
- A Bayesian Credible Interval differs fundamentally from a frequentist Confidence Interval; it allows for a direct probability statement (e.g., "There is a 95% probability that the true parameter lies within this specific range").
- Always translate the numerical summaries back into the scientific context of the original question.

Prior Influence

- A weakly informative prior lets the likelihood (the data) dominate the posterior shape.
- A heavily misspecified, strong prior can pull the posterior away from the true data generating parameter.
- However, as the sample size n increases, the likelihood will continually grow in influence and eventually overwhelm even a stubborn prior.

Week 12: Hidden Markov Models

Motivation and Concept

- Standard Markov models are designed to model transitions between observable states.
- When the underlying states governing a system are unobservable (hidden), but we have access to measurements that depend on these states, we use a Hidden Markov Model (HMM).
- For example, the actual weather (Sunny, Cloudy, Rainy) might be hidden from an observer in a windowless room, but they can observe whether a colleague brings an umbrella to work.

Model Definition and Structure

- An HMM is defined by two linked sequences:
  - **Hidden States (S):** A sequence $S_1, S_2, \dots, S_N$ that strictly follows the Markov property, forming a Markov Chain.
  - **Observations (X):** A sequence of measurements $X_1, X_2, \dots, X_N$ where each observation depends only on the current hidden state.
- The system is fully characterized by two key probability distributions:
  - **Transition Probabilities:** The probability of the system moving from one hidden state to another in the next time step.

$$P(S_t \mid S_{t-1})$$

  - **Emission Probabilities:** The probability of recording a specific observation given the current hidden state.

$$P(X_t \mid S_t)$$

- HMMs are heavily utilized across various domains, including thermodynamics, finance, signal processing, pattern recognition (e.g., speech and handwriting), and bioinformatics.

Mixture Models and Time Dependence

- When modeling sequential count data (such as the number of major earthquakes per year), a baseline approach is to use a simple Poisson distribution.
- **Independent Mixture Model:**
  - Assumes the data is generated by multiple independent intensity periods (e.g., a "high" intensity state and a "low" intensity state) that do not depend on the previous year's state.
  - For a two-component Poisson mixture, the density is parameterized by a mixture proportion γ and state-specific rates λ.

$$f(x) = \gamma p(x; \lambda_1) + (1 - \gamma) p(x; \lambda_2)$$

- **Dependent Mixture Model (HMM):**
  - A more realistic assumption is that the high and low intensity regimes follow a Markov Chain, making the states time-dependent.
  - Extending an independent mixture to an HMM increases the number of parameters to estimate.
  - For a two-state HMM, exactly five parameters must be estimated: one for the initial state probability $P(S_1)$, two for the transition matrix probabilities, and two for the mean emission rates λ_1 and λ_2.

Likelihood and The Expectation-Maximization (EM) Algorithm

- To estimate the parameters of an HMM, we use Maximum Likelihood Estimation.
- The likelihood L of an entire observed sequence is computed as a matrix product:

$$L = \delta P(x_1) \Gamma P(x_2) \ldots \Gamma P(X_N) 1'$$

- In this likelihood equation, δ is the initial state distribution, Γ is the transition matrix, and $P(x_t)$ represents a diagonal matrix of the emission probabilities.
- Because the underlying states are hidden, we cannot maximize this likelihood directly; we must use the Expectation-Maximization (EM) algorithm.
- We define indicator variables for the hidden states: $u_j(t) = 1$ if $X_t = j$ (and 0 otherwise), and $v_{jk}(t) = u_j(t-1)u_k(t)$.
- **The EM Algorithm Steps for HMMs:**

– **E-Step (Expectation):** Compute the expected values of the indicator variables given the full sequence of observed data.

$$E[u_j(t)] = P(X_t = j \mid Y_1 = y_1, \ldots, Y_t = y_t)$$

$$E[v_{jk}(t)] = P(X_{t-1} = j, X_t = k \mid Y_1 = y_1, \ldots, Y_t = y_t)$$

– **M-Step (Maximization):** Maximize the expected log-likelihood using these computed probabilities to update the transition and emission parameters.

Model Selection and Posterior State Decoding

- Competing models (e.g., a single-component Poisson, an independent mixture, and an HMM) can be formally compared using a Likelihood Ratio Test, AIC (Akaike Information Criterion), or BIC (Bayesian Information Criterion).
- Once the optimal HMM is fitted to the data, we use the **forward-backward algorithm** to infer the hidden states.
- This algorithm calculates the posterior probability that the system was in a specific state (e.g., the "high-intensity" regime) at any given time point t, conditioned on the entire observed sequence and the fitted model parameters.
- Posterior decoding is crucial for identifying historical regime shifts, detecting changes in underlying states, and understanding the hidden dynamics that generated the observed sequence.

Complete Exam Quick Reference Table

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Sample Mean	$\bar{x} = \frac{\sum x_i}{n}$	Estimate μ	Centre of data
Sample Variance	$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$	Spread	Uses $n-1$
Sample Std Dev	$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$	Spread in units	Root of variance
Addition Rule	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$	Combine events	Avoid double counting
Complement Rule	$P(A^c) = 1 - P(A)$	At least one questions	Often simplifies
Conditional Prob	$P(B \mid A) = \frac{P(A \cap B)}{P(A)}$	Given info	Order matters
Independence	$P(A \cap B) = P(A)P(B)$	Check independence	Only if unrelated
Bayes Theorem	$P(A \mid B) = \frac{P(B\|A)P(A)}{P(B)}$	Reverse conditional	Common trap
Binomial Mean	$\mu = np$	Expected successes	Fixed n, p
Binomial SD	$\sigma = \sqrt{np(1-p)}$	Spread	Memorise
Poisson Mean	$\mu = \lambda$	Arrivals	Mean = variance
Uniform PDF	$f(x) = \frac{1}{b-a}$	Constant density	Area = probability

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Z Statistic	$Z = \frac{\bar{X}-\mu}{\sigma/\sqrt{n}}$	Mean tests	Known σ
Confidence Interval	$\bar{x} \pm z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$	Estimate mean	Check if μ_0 inside
Shapiro Wilk Test	`shapiro.test(x)`	Normality	H_0: Normal
Levene Test	`leveneTest()`	Compare variances	Robust
Bartlett Test	`bartlett.test()`	Compare variances	Needs normality
One Sample t Test	`t.test(x, mu=...)`	Mean vs constant	Unknown σ
Independent t Test	`t.test(x, y)`	Two groups, unequal/unknown variances	Welch default; no equal variance assumption
Independent t Test (Equal Var)	`t.test(x, y, var.equal=TRUE)`	Two groups, equal variances confirmed	Pooled; only after Levene/Bartlett p > 0.05
Paired t Test	`t.test(x, y, paired=TRUE)`	Before vs after	Uses differences
Proportion Test	`prop.test(x, n)`	Test proportion	Large samples ($n \geq 30$), normal approx
Exact Binomial Test	`binom.test(x, n, p=...)`	Test proportion exactly	Small samples; exact binomial p-value
Chi Square Statistic	$\chi^2 = \sum \frac{(O-E)^2}{E}$	Categorical tests	Large = big difference
Goodness of Fit	`chisq.test(x, p=probs)`	Match distribution	$df = k-1$; requires 80% expected counts ≥ 5
Independence Test	`chisq.test(matrix)`	Relationship test	$df = (r-1)(c-1)$; requires Rule of 5
Wilcoxon Rank Sum Test	`wilcox.test(x, y)`	Compare medians (independent)	Non-parametric alternative to t-test
Wilcoxon Signed-Rank Test	`wilcox.test(x, y, paired=TRUE)`	Compare medians (paired)	Non-parametric alternative to paired t-test
Fisher Exact Test	`fisher.test(matrix)`	Small samples	Use when Rule of 5 violated
Effect Size (Phi)	$\phi = \sqrt{\frac{\chi^2}{n}}$	Strength of association	0.1 small, 0.3 med, 0.5 large
Cohen's d	$d = \frac{\bar{x}_1-\bar{x}_2}{s}$	Effect size for mean differences	0.2 small, 0.5 medium, 0.8 large.

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Likelihood	$L(\theta) = \prod f(x_i\|\theta)$	Parameter estimation	Maximise
Log Likelihood	$\ell(\theta) = \log L(\theta)$	Simplify math	Turns product into sum
MLE Normal Mean	$\hat{\mu} = \bar{x}$	Estimate mean	Same as sample mean
MLE Normal Variance	$\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2$	Estimate variance	Biased
BFGS	`optim(method="BFGS")`	General-purpose MLE optimization	Fast, robust, no second derivatives required
Nelder-Mead	`optim(method="Nelder-Mead")`	Non-smooth likelihoods	Very robust but slower, weak in high dimensions
Negative Log-Likelihood	$-\sum \log f(x_i \mid \theta)$	Convert maximization to minimization	R minimizes by default
Convergence Check	`fit$convergence == 0`	Verify optimizer success	0 indicates successful convergence
Equivariance (MLE)	If $\hat{\theta}$ is MLE, then $g(\hat{\theta})$ is MLE of $g(\theta)$	Transformations of parameters	Core theoretical property
LRT Statistic	$\Lambda = -2[\ell(\hat{\theta}_0) - \ell(\hat{\theta})]$	Compare nested models	Based on log-likelihood difference
LRT Distribution	$\Lambda \sim \chi^2_{df}$	Compute p-values	df equals number of restrictions
Profile Confidence Intervals	`confint(fit)`	Construct CIs via LRT	Does not rely on normal approximation
Linear F-Statistic	$H_0 : \beta_1 = \cdots = \beta_p = 0$	Overall model significance	Tests if at least one predictor matters
GLM Setup	$g(\mu_i) = \eta_i$	Linking mean to predictors	Connects distribution to linear equation
Logit Link	$\log\left(\frac{p}{1-p}\right) = \eta$	Logistic regression link	Bounds predictions to $[0, 1]$
Odds	$\text{odds} = \frac{p}{1-p}$	Probability to Odds	Ratio of success to failure
Probability from Odds	$p = \frac{\text{odds}}{1+\text{odds}}$	Odds to Probability	Reverses the odds calculation
Odds Ratio	e^{β_j}	Interpreting logistic coefficients	> 1 increases odds, < 1 decreases odds

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Model Comparison (LRT)	`anova(mod1, mod2, test="Chisq")`	Full vs Reduced model	Tests if added variables improve fit
GLM Fitting Algorithm	Iteratively Reweighted Least Squares (IRLS)	Optimization in `glm()`	More efficient than standard `mle()`
Formula: +	`y ~ x1 + x2`	Adding main effects	Adds independent predictors
Formula: *	`y ~ x1 * x2`	Main effects + interaction	Shortcut for `x1 + x2 + x1:x2`
Formula: :	`y ~ x1:x2`	Interaction term only	The effect of one depends on the other
Model Deviance	$D = -2[l(\hat{\beta}) - l(\text{saturated})]$	Measure GLM fit	Like RSS for GLMs
AIC	$AIC = -2l(\hat{\beta}) + 2k$	Compare non-nested models	Lower is better
BIC	$BIC = -2l(\hat{\beta}) + k \log n$	Compare non-nested models	Stronger penalty than AIC
Poisson Regression	`glm(y ~ x, family=poisson)`	Count data	Mean = Variance (λ)
Offset (Rate Modelling)	`offset(log(exposure))`	Model rates not totals	Constrains exposure coefficient to 1
Dispersion Statistic	$\hat{\phi} = \frac{\text{Residual Deviance}}{\text{Residual df}}$	Check overdispersion	$\hat{\phi} \geq 3$: discard Poisson
Quasi-Poisson	`glm(y ~ x, family=quasipoisson)`	Overdispersed counts	No AIC; SEs multiplied by $\sqrt{\hat{\phi}}$
Negative Binomial	`glm.nb(y ~ x)` from `MASS`	Overdispersed counts	Has AIC; variance $= \mu + \mu^2/r$
Gamma GLM	`glm(y ~ x, family=Gamma(link="log"))`	Positive right-skewed data	Variance $\propto \mu^2$
Rate Ratio (Poisson/Gamma)	$e^{\hat{\beta}_j}$	Interpret log-link coefficients	> 1 increases rate/mean, < 1 decreases
Dummy Coding	`as.factor(x)` in formula	Categorical predictors	R uses $n-1$ bits; avoids multicollinearity
KDE	$\hat{f}(x) = \frac{1}{nh} \sum K(\frac{x - x_i}{h})$	Non-parametric density estimation	Kernel + bandwidth; must integrate to 1
Silverman's Rule	$h \approx 1.06\, \sigma\, n^{-1/5}$	Default bandwidth	Assumes Normal; R: `bw.nrd0`
Cross-Validation (BW)	`bw.ucv` / `bw.bcv`	Data-driven bandwidth	Minimises integrated square error

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Epanechnikov Kernel	Optimal efficiency kernel	KDE kernel choice	Most efficient; all common kernels $> 90\%$
`density()`	`density(x)`	Compute KDE in R	Uses FFT; $O(m \log m)$
Bayes Classifier	$P(C \mid X) = \frac{f(X\|C)P(C)}{f(X)}$	Non-parametric classification	Estimate $f(X \mid C)$ via KDE
2D KDE	`kde2d()` from `MASS`	Bivariate density	Separate bandwidths per dimension
Collinearity	Correlated predictors	Unstable coefficients	Drop one dummy level as baseline
Mixture Density	$f(x) = \sum p_j f_j(x)$	Model sub-populations	Weights sum to 1
EM Responsibility	$\gamma_i = \frac{\lambda f_1(x_i)}{\lambda f_1(x_i)+(1-\lambda)f_2(x_i)}$	E-step (prob of comp 1)	Fractional component membership
Mixture Complexity	$p = 6k - 1$	Parameter count	For bivariate Normals
Markov Property	$P(X_t \mid X_{t-1}, \dots) = P(X_t \mid X_{t-1})$	Future depends on present	Memoryless assumption
n-Step Transition	P^n	Multi-step probability	Multiply transition matrix
State Evolution	$p_n = p_0 P^n$	Find future distribution	Row vector $\times$ matrix
First Passage Time	$h_{ij}(n) = \sum_{k \neq j} p_{ik} h_{kj}(n-1)$	Time to hit state first time	Recursive formula
Expected Passage	$E[T] = \sum n \cdot h_{ij}(n)$	Avg time to reach state	Weighted sum of times
Bayesian Prior	$p(\theta)$	Initial belief about parameter	Chosen before seeing data
Bayesian Likelihood	$p(y \mid \theta)$	Evidence from observed data	Probability of data given parameter
Bayesian Posterior	$p(\theta \mid y) = \frac{p(y\|\theta)p(\theta)}{p(y)}$	Updated belief after data	Combines prior and likelihood
Conjugate Prior	Prior where posterior is same family	Simplifies computation	Allows closed-form solutions
Beta-Binomial Model	Beta(α, β) prior, Binomial likelihood	Model proportions/probabilities	Conjugate pair
Posterior Parameters (Beta-Binomial)	Beta$(\alpha + y, \beta + n - y)$	Updated shape parameters	y = successes, n = trials

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Maximum A Posteriori (MAP)	$\frac{\alpha-1}{\alpha+\beta-2}$ for Beta	Mode of posterior distribution	Regularized MLE with prior
Credible Interval	Posterior quantiles (e.g., 5th, 95th percentiles)	Bayesian confidence interval	Direct probability interpretation
Marginal Likelihood	$p(y) = \int p(y \mid \theta)p(\theta)\, d\theta$	Normalizing constant in Bayes Theorem	Intractable for non-conjugate models
MCMC (Markov Chain Monte Carlo)	Construct ergodic chain with target posterior as stationary distribution	Sample from intractable posteriors	Convergence guaranteed by ergodic theorem
Metropolis-Hastings Algorithm	Accept-reject mechanism with ratio $\alpha = \frac{f(\theta^*)}{f(\theta^{(t)})} \cdot \frac{g(\theta^{(t)}\mid\theta^*)}{g(\theta^*\mid\theta^{(t)})}$	Generate MCMC sequence	Satisfies detailed balance
Proposal Tuning	Optimal step size yields acceptance rate 20%-50%	Tune proposal standard deviation	Too small = slow exploration; too large = high rejection
Trace Plot	Visualize parameter values across iterations	Assess MCMC mixing	Should look like stationary white noise
Burn-in	Discard initial transient iterations	Remove bias from starting value	Typically 10%-20% of total chain
Autocorrelation Function (ACF)	Correlation between samples at different lags	Check mixing quality	Fast decay = good mixing
Effective Sample Size (ESS)	Equivalent number of independent samples	Assess MCMC efficiency	Low ESS requires longer chain
Gelman-Rubin Statistic	$\hat{R} \approx 1$ indicates convergence	Compare multiple chains	$\hat{R} > 1.1$ indicates non-convergence
Prior Predictive Check	Simulate data from prior before fitting	Validate prior specification	Ensure prior covers observed data range
Log-Posterior	Sum of log-likelihood and log-prior	Numerically stable computation	Avoid underflow in software
Hidden Markov Model (HMM)	Hidden states + observations; states follow Markov property	Model unobservable states	Observations depend only on current state
HMM Transition Probabilities	$P(S_t \mid S_{t-1})$	Probability of state transitions	Part of HMM specification
HMM Emission Probabilities	$P(X_t \mid S_t)$	Probability of observation given state	Links observations to hidden states

Concept / Test	Formula or R Function	Use Case	Key Exam Notes
Independent Mixture Model	$f(x) = \gamma p(x; \lambda_1) + (1-\gamma) p(x; \lambda_2)$	Multiple regimes without time dependence	Baseline model without Markov structure
Dependent Mixture (HMM)	Mixture components follow Markov Chain	Time-dependent regime switching	More realistic than independent mixture
HMM Likelihood	$L = \delta P(x_1) \Gamma P(x_2) \ldots \Gamma P(X_N) 1'$	Probability of entire observation sequence	Product of transition and emission matrices
EM Algorithm for HMMs	E-Step: compute expected state indicators; M-Step: update parameters	Optimize HMM parameters	Handles hidden state uncertainty
Forward-Backward Algorithm	Calculate posterior probability of state at time t given full sequence	Infer hidden states	Posterior decoding
Posterior Decoding	Extract most probable hidden state sequence	Identify regime shifts	Understand hidden dynamics